

4. General Design and Architecture Choices

4.1 - Design Requirements and Principles

The architecture follows the requirements in section 1.5. It must make source charges traceable, validate data before publication and serve consistent indicators to users with different access rights. The distinction between these requirements and the available tools matters because the local POC, cloud prototype and possible enterprise deployment have different operating conditions.

Functional drivers

- ingest recurring FOCUS cost and usage data;
- preserve received evidence and its provenance;
- validate a versioned data contract before canonical publication;
- publish consistent FinOps data products for Inform, Optimize and Operate;
- support dashboard users and authorized analytical consumers;
- reconcile row counts and financial measures across transformations.

Quality and governance drivers

- explicit ownership and definitions;
- controlled schema evolution;
- record-level and batch-level lineage;
- reproducible processing and validation;
- visibility of rejected, missing or unallocated information;
- separation between canonical data and derived analytical products.

Security drivers

- least-privilege and read-only analytical access;
- isolation of managerial or domain perimeters;
- protection of direct SQL access in addition to dashboard filtering;
- no credentials committed to source control.

Operational drivers

- restart and recovery information;
- observable freshness and execution status;

- maintainability by the platform and FinOps roles;
- portability between the local validation environment and the target cloud environment where feasible.

Project constraints

The design must remain achievable within the PFE schedule and available licences. Confidentiality limits direct use of enterprise billing data. Only a restricted anonymized source is available. The local DuckDB POC, the Databricks/GCP DEV and PROD workspaces, and a possible enterprise Azure target provide different levels of evidence.

4.2 - General Platform Design (Logical HLD)

System boundary

The system begins with cost and usage datasets delivered by a billing-data producer and ends with governed information exposed to FinOps, managers and authorized analysts. Provider billing generation, enterprise accounting, resource remediation and production identity administration remain external responsibilities.

Logical flow

```
Cost and usage sources
      v
Evidence preservation
      v
Ingestion and lineage
      v
Contract and quality validation
      v
Canonical cost and usage data
      v
Dimensional and subject-oriented data products
      v
Governed dashboards and analytical access
```

Quality, reconciliation, security, metadata and observability apply across the complete flow.

Logical components

Component	Responsibility
Source interface	Receive periodic cost and usage data with identifiable origin and period
Evidence store	Preserve the received representation without hidden business transformation
Ingestion service	Register the batch and add technical lineage
Contract gate	Reject or quarantine incompatible structure and invalid mandatory values
Canonical repository	Provide the authoritative standardized FinOps representation
Analytical model	Organize detailed charges by time, scope, resource, service, price and commitment
Data-product layer	Publish stable subjects and common measures for consumption
Consumption layer	Provide role-appropriate dashboards and analytical access
Operational control	Track executions, freshness, failures, reconciliation and recovery

Architectural principles

- The immutable evidence layer and the canonical repository have different responsibilities.
- Validation occurs before data becomes authoritative.
- Derived products do not become independent sources of truth.
- Consumers use stable products rather than storage-specific implementation details.
- Access control applies at every exposed interface.
- Claims of reliability require reproducible evidence, not only an architecture diagram.

Operating model: centralized production and distributed consumption

The proposed operating model gives managers access to their cost scope and authorized analysts access to further analysis. The Data Platform retains shared technical capabilities; FinOps owns business definitions and product validation; domains consume only their authorized scope. These responsibilities describe the target design and require organizational confirmation.

Here, “data factory” denotes a centralized production organization, not the Azure Data Factory service. Centralized production can coexist with autonomous access. The issue under study is dependency on repeated requests for changes.

Data Mesh combines domain ownership, data as a product, self-service infrastructure and federated governance. Its autonomy extends to producing teams, beyond report consumption. Physical data location alone does not define it.

Source: Dehghani, Data Mesh Principles and Logical Architecture (2020).

The target adopts product-oriented practices-definitions, quality, ownership and stable interfaces-without autonomous production by several domains or full federated governance. It is therefore not described as a complete Data Mesh. Chapter 9 will revisit this choice against the evidence collected.

This section describes what the platform must do without assigning those responsibilities to products. Section 4.3 compares the documented technology options and identifies the tools used in the prototypes. Section 4.4 maps the logical flow to deployed components; section 4.5 explains the data model and the analytical products.

4.3 - Technology Evaluation and Selection

4.3.1 Evaluation basis

The project's initial architecture evaluation considered Microsoft Fabric, Azure Databricks and a modular Azure PaaS solution against requirements for ingestion, contract validation, governed analytical products and reporting. It also listed maintainability, security, cost, available licences and feasibility within the PFE as criteria. The evaluation was qualitative: it contained neither weighted scores nor a measured total cost for the three candidates. It can explain which risks were considered, but it cannot support a numerical ranking.

4.3.2 Platform alternatives and implemented scope

Fabric groups data preparation, storage, SQL and Power BI within one service environment. The initial record treats this integration as a possible advantage and flags capacity and licensing as questions. Azure Databricks offers a lakehouse processing path with Unity Catalog and SQL serving, but the proposed Azure architecture would also require storage, identity and reporting integration. A modular Azure solution would make service boundaries explicit while increasing the number of components to configure and operate. These are design observations from the initial evaluation, not benchmark results.

The implementation subsequently converged on Databricks with GCP object storage for the active cloud prototype. Its code and catalog organization are described in sections 4.4 and 5.3; execution evidence is assessed in section 7.4. This implementation choice establishes what was built; the available decision record does not isolate the commercial, access or scheduling reason for the move from the Azure-first options. The thesis therefore does not present the GCP prototype as an approved enterprise Azure deployment or infer that it outperformed Fabric on cost.

4.3.3 Storage and processing choices

The independent generator publishes Parquet files as the shared RAW input. Apache Parquet specifies a column-oriented file format for storage and retrieval. The Databricks pipeline publishes managed Delta tables in Bronze, Silver and Gold, where month-level replacement and table history are required. Delta Lake extends Parquet data files with a transaction log; the two formats therefore serve different roles in this design rather than being mutually exclusive alternatives. A replacement is atomic for an individual Delta table operation, while the entire multi-table pipeline needs audit and recovery logic.

Source: Apache Parquet, Overview; Databricks, What is Delta Lake in Databricks?.

Python and PyArrow support deterministic file generation and schema checks. SQL defines the Gold model and datamarts, and PySpark coordinates their execution in Databricks. YAML records the versioned data contract, while DBML documents the dimensional design. DuckDB serves the frozen local POC; its results are evidence for that environment only.

4.3.4 Reporting decision and remaining comparison

The POC uses Streamlit with DuckDB datamarts, whereas the cloud application is designed to query Databricks through a SQL Warehouse. The existing Power BI dashboard remains the professional reference system. The repository demonstrates local Streamlit execution, but it does not contain a controlled comparison of these reporting options or validated Row-Level Security for the new application. The selection can therefore be defended for the implemented prototype and its reproducible tests; an enterprise adoption recommendation would also require access, performance and operating-cost evidence.

4.3.5 Technology choices by function

The following table summarizes the tools present in the two implementations. The local POC is a separate experiment, not the DEV environment of the cloud platform. DEV and PROD use the same Databricks/GCP architecture with separate business catalogs. The table reports implementation choices and their role; it is not a scored comparison of every tool named during early exploration.

Function	Local POC	Cloud DEV and PROD	Rationale and evidence boundary
Source files	Shared synthetic Parquet files on the local filesystem	The same published Parquet history in GCS, exposed through an external Unity Catalog volume	A common input supports comparison; publication and cloud ingestion need separate checks.

Function	Local POC	Cloud DEV and PROD	Rationale and evidence boundary
Data processing	Python/Pandas and local notebooks	PySpark for ingestion and contract processing; versioned SQL for Gold and datamarts	The cloud path uses distributed Spark execution, but no performance advantage follows without a comparable benchmark.
Processed tables	Local Delta layers and a DuckDB analytical warehouse	Managed Delta tables in Bronze, Silver, Gold and datamarts	Delta supports table-level replacement and history; DuckDB supplies local analytical tests.
Orchestration	Explicit local commands and notebook execution	Databricks Jobs coordinating notebooks and validation tasks	The cloud Jobs encode task dependencies; an automatic schedule does not by itself prove end-to-end operation.
Governance and audit	Local contract checks and test outputs	Unity Catalog namespaces and permissions; <code>finops_ops.audit</code> for operational history	Catalog organization is implemented, while effective access and recovery still require run-level tests.
Analytical access	DuckDB queries over local datamarts	SQL Warehouse queries over PROD datamarts and OPS audit	The Warehouse serves the application; it is not the compute engine used by every batch task.
Visualization	Local Streamlit application	Separate Streamlit Databricks App	The local interface is tested; cloud page behavior and permissions need their own evidence. Power BI remains the existing professional reference system.

The initial evaluation documents Fabric, Azure Databricks and modular Azure PaaS as platform alternatives. It does not provide controlled scores for Colab, Airflow, Tableau or every other tool that could be placed in a category. Adding such products to this table would imply an evaluation that the project did not perform.

4.4 - Selected Physical Architecture

Section 4.2 defines the required functions, and section 4.3 explains the chosen technologies. This section places the implemented components in their actual storage, catalog, processing and access boundaries. It answers where data resides and how the components connect; it does not repeat the technology comparison or detail every table, which is the purpose of section 4.5.

4.4.1 System boundary and implementation tracks

The implementation separates three responsibilities. `FinOps Data Generator` produces a single FOCUS-based synthetic dataset, independently of any analytical engine. `FinOps Data Platform - POC` preserves the earlier DuckDB/Streamlit experiment and its January 2025 local results. `FinOps Cloud Data Platform` is the active Databricks/GCP implementation. This division preserves the value of the local validation without implying that DuckDB and Spark execute identical physical code or that local results validate cloud operation.

The cloud platform receives detailed Daily and monthly Parquet files in a shared RAW namespace, transforms them through Bronze and Silver Delta tables, builds a Gold dimensional model, and publishes subject-oriented datamarts. Its read-only application consumes the datamarts and operational audit through a SQL Warehouse. The architecture is an implemented PFE prototype with planned extensions; it is not a claim of deployment into Technip Energies production systems.

```
Independent generator -> GCS RAW Parquet -> finops_raw.landing
                        └─ DEV -> Bronze -> Silver -> Gold -> datamarts
                        └─ PROD -> Bronze -> Silver -> Gold -> datamarts
DEV and PROD runs -> finops_ops.audit
PROD datamarts + audit -> SQL Warehouse -> App
```

4.4.2 Source authority, catalogs and data lifecycle

`finops_raw.landing` exposes the same GCS objects to both environments. DEV business tables reside in `finops_dev`; PROD business tables reside in `finops_prod`. Each contains Bronze, Silver, Gold and datamart schemas. The separate `finops_ops.audit` catalog has five operational tables and records an explicit `environment` on each row. This arrangement permits independent data publication while maintaining a shared operational history. It also makes an environment filter essential in every audit lookup and write.

The source lifecycle follows financial meaning. Daily files populate an open month provisionally. The detailed monthly billing file becomes authoritative after close and replaces that month's Silver and Gold contents. The two source types are never added together in the fact table. A monthly replacement is therefore a correction of the canonical state rather than another batch of cost on top of Daily activity.

Automatic RAW archival is disabled in the common configuration. The earlier design would have moved processed sources after close, but a DEV close could then remove a file needed for independent PROD processing. Keeping the RAW objects allows both environments to consume the same input and preserves the evidence used for replays. This choice has a storage and lifecycle cost: a future retention rule must identify every consumer and the audit period before moving or deleting any source. The archival module remains in the codebase as an optional capability, not an active lifecycle step.

4.4.3 Compute configuration boundary

The repository separates common processing logic from compute-specific deployment artifacts. `platform/common` contains shared notebooks and SQL. `platform/serverless` and `platform/classic_compute` contain the respective setup and Job adapters. Both scenarios target the same logical RAW-to-datamart flow, while catalog storage configuration, compute lifecycle and billing attribution differ. This separation allows the implementation to change compute without redefining the data contract or analytical grain.

This architecture section records the physical components and their interfaces, not the chronology of workspace decisions or a claim that either compute mode is more economical. Chapter 7 records execution evidence and chapter 8 examines the later location and compute choices as an operating-cost case study.

4.4.4 Analytical and consumption layers

Silver retains the canonical FOCUS representation after contract validation. Gold and the fourteen datamarts are stored separately in each business catalog. Versioned SQL defines and refreshes these products; Python orchestrates execution and supplies validated environment identifiers. Some datamarts read central Silver fields absent from the Gold fact, while others read Gold. Section 4.5 specifies the tables, their grain and the business questions they are designed to address. A full datamart refresh favors simple publication in the current version at the expense of extra work after each monthly replacement.

The frozen local dashboard demonstrated that eleven of fourteen local datamarts support its views. The active Databricks App is a separate implementation designed to read PROD datamarts and OPS audit in read-only mode. Its service principal is intended to receive only the Unity Catalog and Warehouse permissions required for that use. Actual effective permissions, direct SQL access and Row-Level Security require their own tests; interface behavior alone cannot establish them.

4.4.5 Architectural trade-offs and evidence boundary

The design prioritizes a single source history, explicit environment isolation, a canonical Silver layer and reproducible SQL products. It accepts duplicated DEV/PROD data, materialized datamarts and retained RAW objects to make testing, promotion and audit easier. These choices incur storage, refresh and operational costs that chapter 8 must quantify. Chapter 7 must establish which cloud paths ran successfully and how they compare with the POC. The existence of two compute configurations alone cannot establish a cost or latency advantage.

4.5 - Data Architecture and Dimensional Modeling

4.5.1 Canonical record and analytical grain

The data architecture separates three purposes. RAW retains the received Parquet evidence; Bronze adds technical lineage without changing source business values; and Silver contains the FOCUS representation after the versioned contract has accepted it. Gold reorganizes that canonical record for recurring analysis, while datamarts answer defined questions at coarser grains. Neither Gold nor a datamart becomes an independent source of cost truth. The same logical model is instantiated in separate DEV and PROD catalogs; the frozen local POC uses a related analytical design but a different engine.

The Gold fact, `fact_finops_cost_usage`, represents one FOCUS cost or usage charge line. It retains `billing_month` so that an authoritative monthly bill can replace the provisional Daily-fed month without adding both sources together. Its technical key is deterministic, using source lineage and logical row position; dimensional foreign keys connect a charge to its time, scope, resource, service, SKU, location, pricing and charge type. This grain permits reconciliation back to Silver and prevents a monthly summary from obscuring individual adjustments. It does not, by itself, establish the financial meaning of a difference between cost columns.

4.5.2 Gold table inventory and relationships

The cloud SQL defines the following Gold tables. This inventory gives each name and responsibility. A full column-by-column DDL listing is not required to interpret the model in this chapter.

Table	Analytical responsibility
<code>fact_finops_cost_usage</code>	Charge-line measures, quantities, billing month and dimensional keys
<code>dim_date</code>	Reusable calendar attributes for the fact's dates
<code>dim_billing_scope</code>	Billing accounts, subaccounts, customers and cost-center attributes
<code>dim_resource</code>	Resource, resource group and application/owner attributes
<code>dim_service</code>	Service, provider, publisher and reseller attributes
<code>dim_sku</code>	SKU, meter, offer and term attributes
<code>dim_location</code>	Region and availability-zone attributes
<code>dim_pricing</code>	Pricing category, unit and currency attributes
<code>dim_commitment_discount</code>	Commitment identifiers and categories

Table	Analytical responsibility
<code>dim_charge_type</code>	Charge category, subcategory and frequency
<code>dim_tag</code>	Normalized tag key/value pairs
<code>bridge_resource_tag</code>	Many-to-many association between resources and tags

The model has ten dimensions. The fact links directly to nine of them; tags are reached through `dim_resource`, `bridge_resource_tag` and `dim_tag`. The model's billing-scope and resource dimensions currently use Type 1 updates. Their validity columns do not prove that every past attribute value is preserved as SCD2 history. The source dataset lacks a native charge identifier and availability zone, so the implementation generates a deterministic charge identifier and uses `Unknown` for the unavailable zone. These are model limitations, not general properties of FOCUS.

4.5.3 Datamarts and business questions

The datamarts are analytical products derived from the canonical representation. The question column states their intended use, not a measured benefit or a claim that every cloud table has been independently validated. The paragraph after the table distinguishes products derived from Gold from those reading central Silver directly.

Datamart	Business or control question supported
<code>dm_monthly_billing</code>	What is the billed cost for each billing month?
<code>dm_daily_billing</code>	How does billed cost vary by charge-start date?
<code>dm_cost_by_scope_service_month</code>	Which cost centers, customers and services account for monthly billed cost?
<code>dm_top_services</code>	Which service categories and names have the highest billed cost in the loaded data?
<code>dm_top_resources</code>	Which resources have the highest billed cost in the loaded data?
<code>dm_cost_by_charge_type</code>	How is monthly billed cost distributed across charge categories and frequencies?
<code>dm_sku_cost</code>	Which SKUs and meters account for the highest billed cost?
<code>dm_savings_monthly</code>	How do monthly List, Contracted and Effective Cost totals differ?
<code>dm_executive_summary_monthly</code>	What are the monthly cost totals, charge-line count, resource count and service count?

Datamart	Business or control question supported
<code>dm_top_resources_monthly</code>	Which resources drive billed cost within a selected billing month?
<code>dm_data_quality_monthly</code>	How many critical nulls and ingestion batches appear in each Silver month?
<code>dm_cost_by_resource_group_month</code>	What are monthly billed cost and resource count by resource group?
<code>dm_cost_by_subscription_month</code>	What are monthly billed cost and resource count by subaccount or subscription?
<code>dm_cost_by_application_owner_month</code>	How are monthly billed cost and resource count attributed to applications and owners?

`dm_savings_monthly`, `dm_executive_summary_monthly` and `dm_data_quality_monthly` read the central Silver representation because they use FOCUS fields or technical metadata not all held in the Gold fact. The remaining datamarts read Gold. The differences named as “savings” in the SQL are arithmetic cost-column comparisons; they do not establish realized savings without eligibility rules, charge-category analysis and business validation. Section 5.3.3 describes how the cloud model and datamarts are built, while chapters 6 and 7 examine their analytical meaning and validation evidence.

References

Apache Parquet. (n.d.). *Overview*. <https://parquet.apache.org/docs/overview/> (accessed 29 September 2026).

Databricks. (n.d.). *What is Delta Lake in Databricks?* <https://docs.databricks.com/gcp/en/delta/> (accessed 29 September 2026).

Dehghani, Z. (2020, 3 December). *Data Mesh Principles and Logical Architecture*. MartinFowler.com. <https://martinfowler.com/articles/data-mesh-principles.html> (accessed 29 September 2026).